ICML’23 Summary

(Visual RL flavor)

Presentor: Hojoon Lee

Contents

Representation Learning for RL

- Single-View Image
- Multi-View Image
- Video (Stream of Images)
- Demonstration (Video + Action)
- Trajectory (Video + Action + Reward)

Plasticity

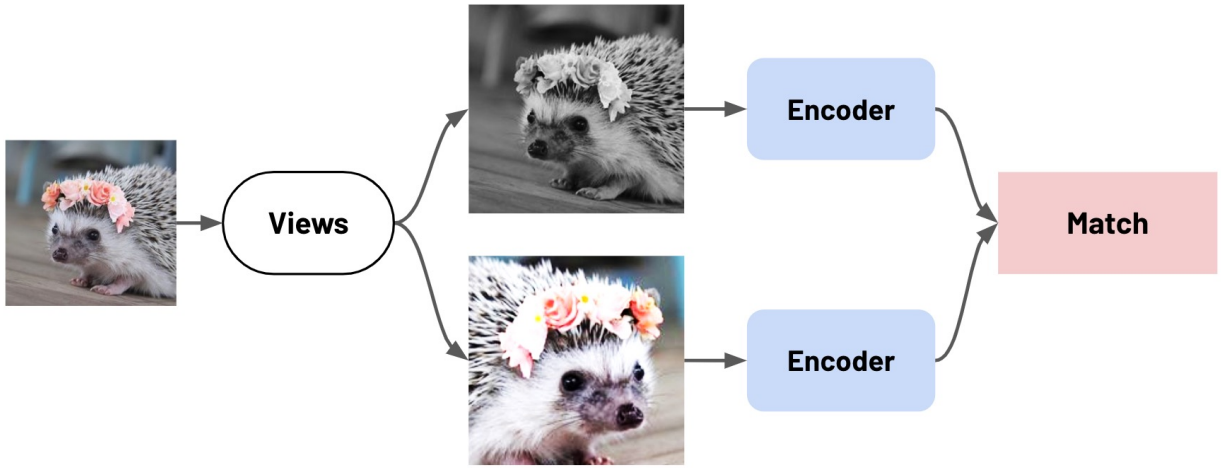
Miscellaneous

Representation Learning for RL

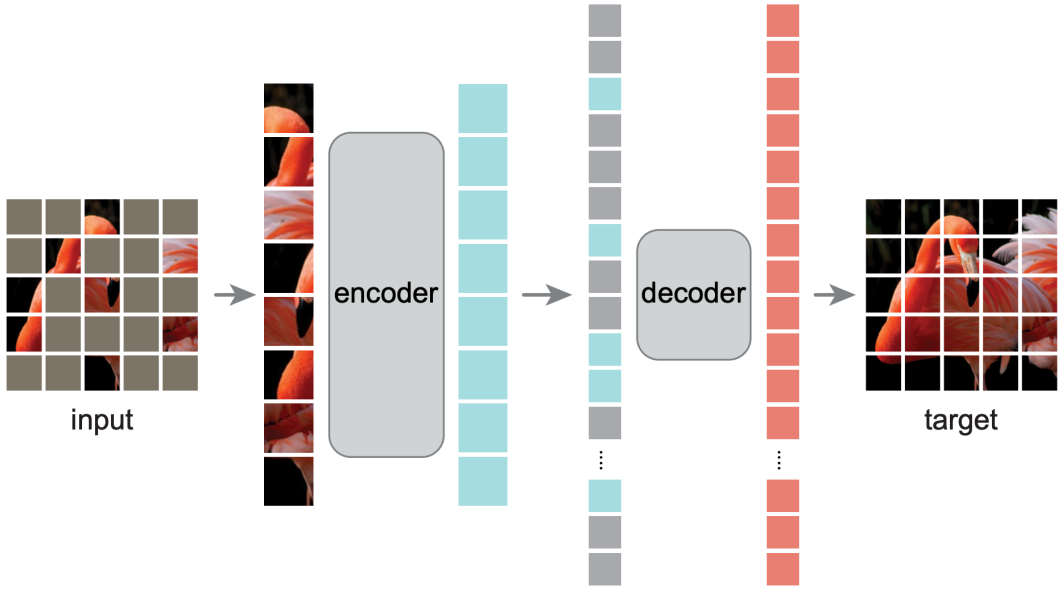
Preliminary: Self-Supervised Learning from Image

Invariance vs Generation

- Let X be data, $Z(X)$ be representation.
- Let I denotes mutual information of two random variables.
  - **Invariance:** Maximize $I(Z(X_1);Z(X_2))$ where X_1, X_2 are invariant data.
  - **Generation:** Maximize $I(Z(\bar{X});X)$ where $\bar{X}$ is perturbed version of X.



Invariance

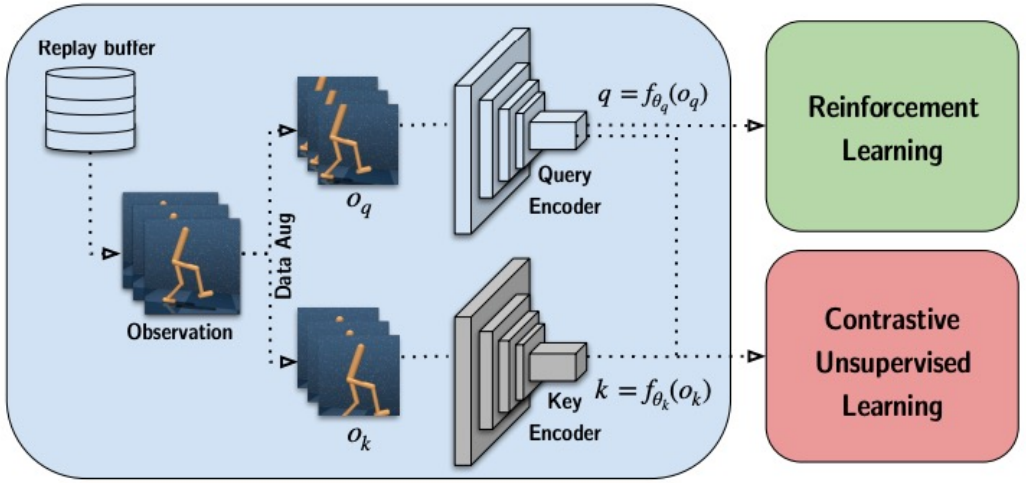


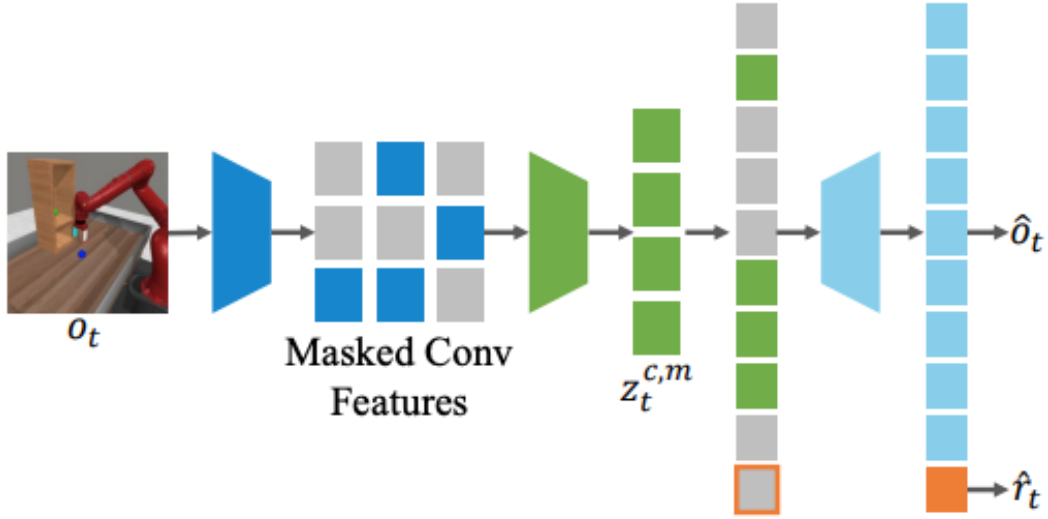
Generative

Learning from Single-View Image

CURL (Invariance) vs MAE (Generation)

- Let X be data, $Z(X)$ be representation.
- Let I denotes mutual information of two random variables.
  - **Invariance:** Maximize $I(Z(X_1);Z(X_2))$ where X_1, X_2 are invariant data.
  - **Generation:** Maximize $I(Z(\bar{X});X)$ where $\bar{X}$ is perturbed version of X.





[1] CURL: Contrastive Unsupervised Representations for Reinforcement Learning. Srinivas et al., ICML 2020.

[2] MWM: Masked World Models for Visual Control. Seo et al., CoRL 2022.

Learning from Single-View Image

Efficient RL via Disentangled Environment and Agent Representations

- Use segmentation mask to disentangle agent from the environment.
- Question: Do we need to generate segmentation mask only to the agent?

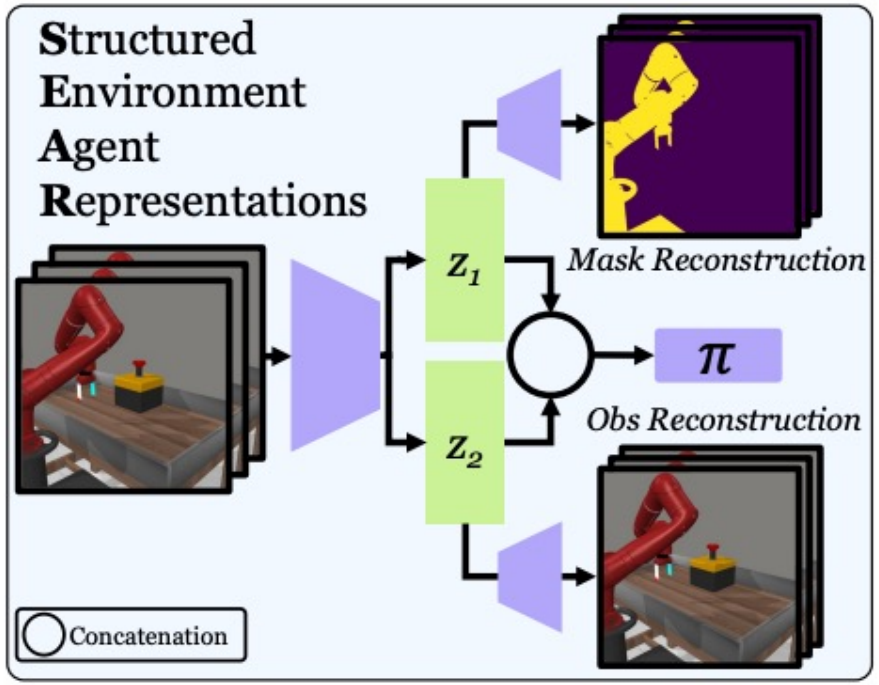


Figure 2: Overview of SEAR. We use the mask of the agent to learn structured representations.

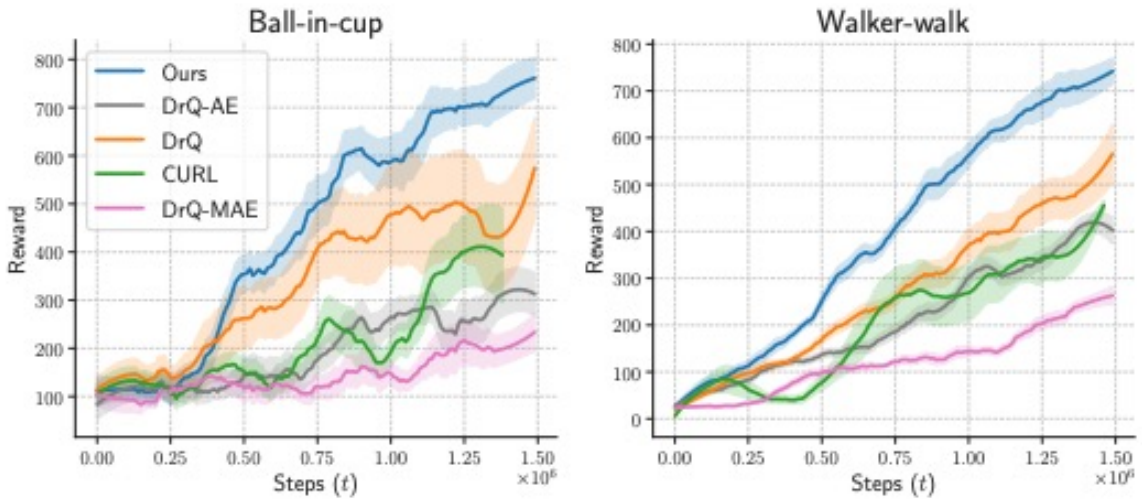


Figure 10: **Distracting Control:** Reward for different methods, SEAR outperforms all the baselines.

Learning from Single-View Image

Efficient RL via Disentangled Environment and Agent Representations

- Does object centric pretraining improves performance on object-centric tasks?
- Object centric reconstruction = unsupervised segmentation

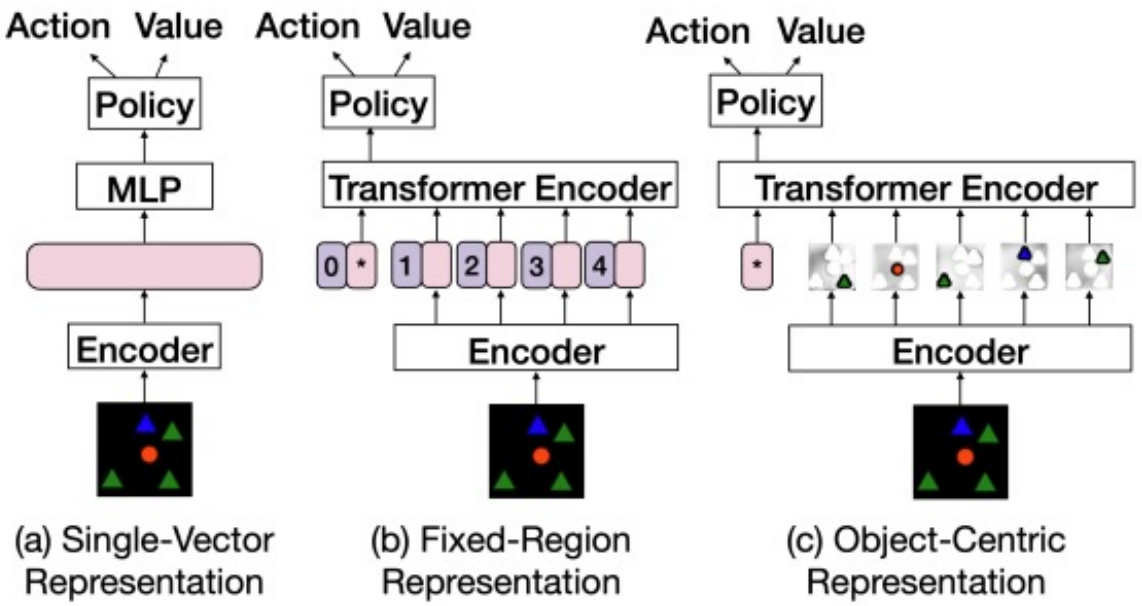
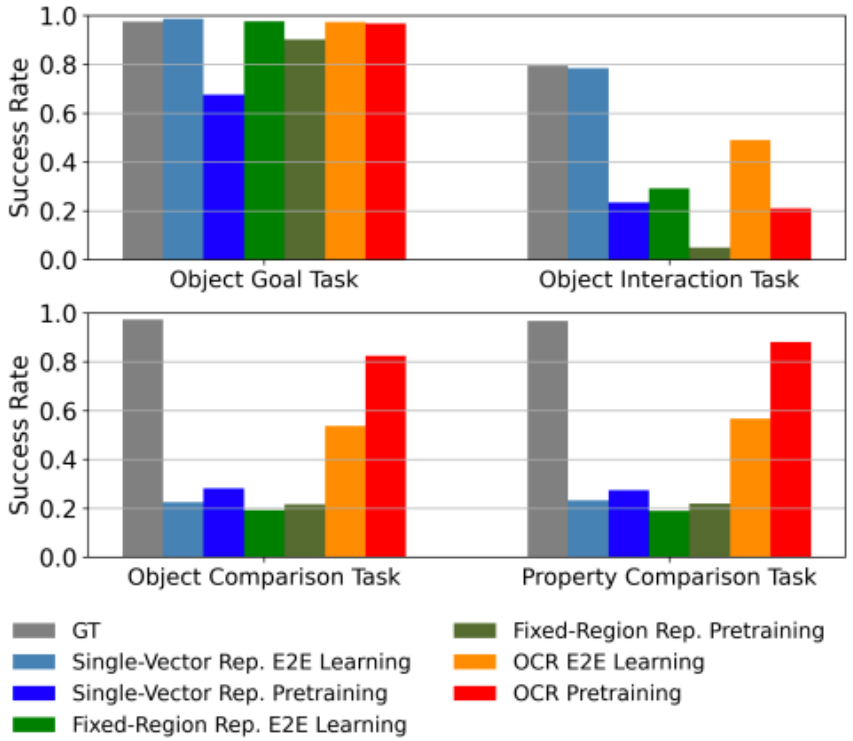


Figure 1: The model architectures for representation types.

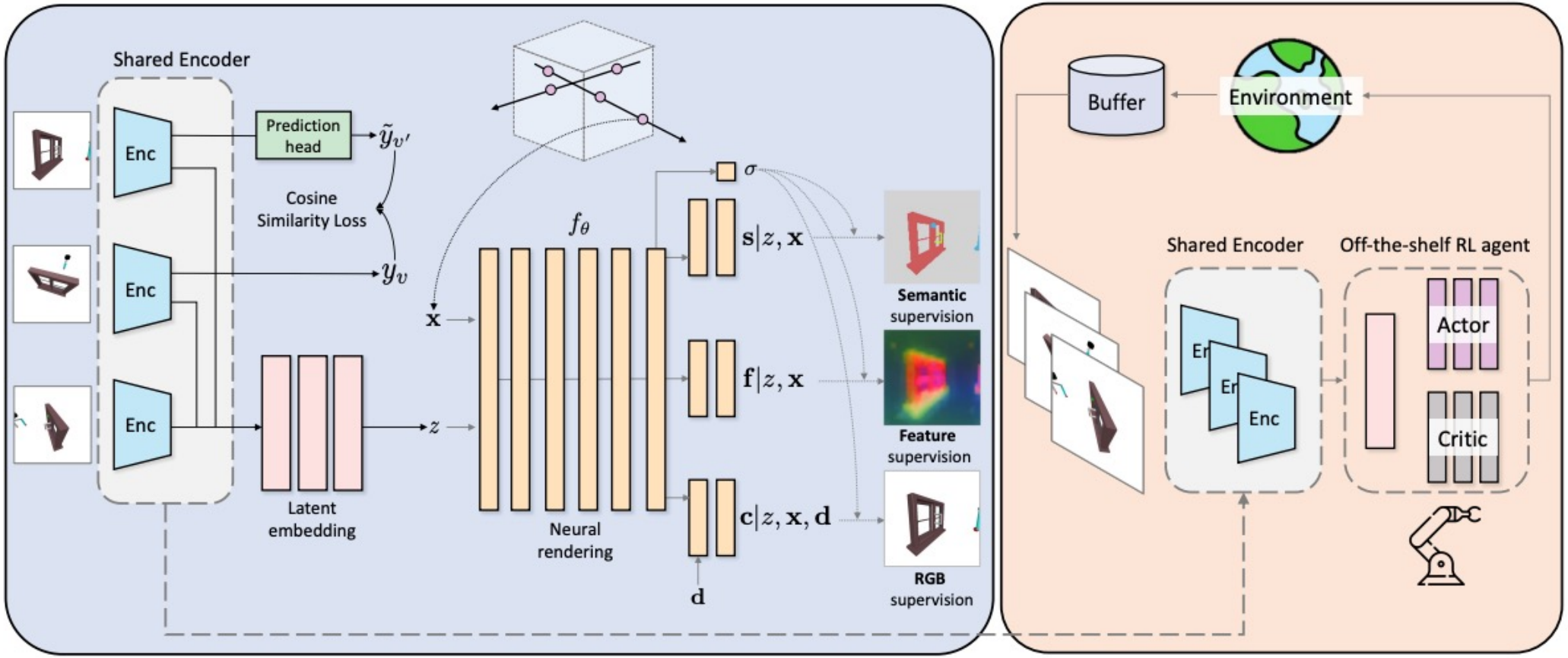


[3] Object-Centric Learning with Slot Attention. Locatello et al., NeurIPS 2020.

Learning from Multi-View Image

SNeRL: Semantic-aware Neural Radiance Fields for Reinforcement Learning

- NeRF-based Generation + Knoweldge Distillation (DINO)



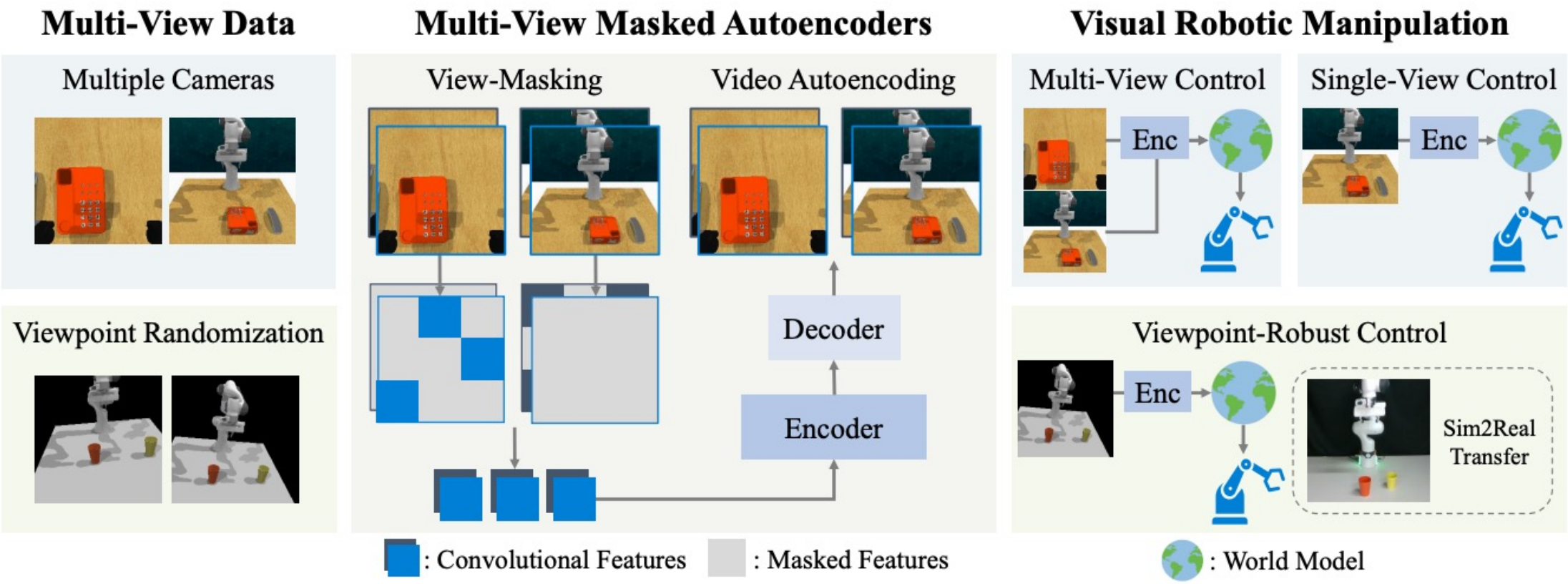
Stage 1. Pre-train multi-view encoder with semantic-aware NeRF

Stage 2. Train downstream off-the-shelf RL agent

Learning from Multi-View Image

MVMWM: Multi-View Masked World Models for Visual Robotic Manipulation

- Expand Masked Auto Encoding to Multi-View Setup



Learning from Multi-View Image

F2C:Information-Theoretic State Space Model for Multi-View RL

- Challenge: Some views are randomly missing.
- Key Idea: Learn the feature extractor robust to missing views.

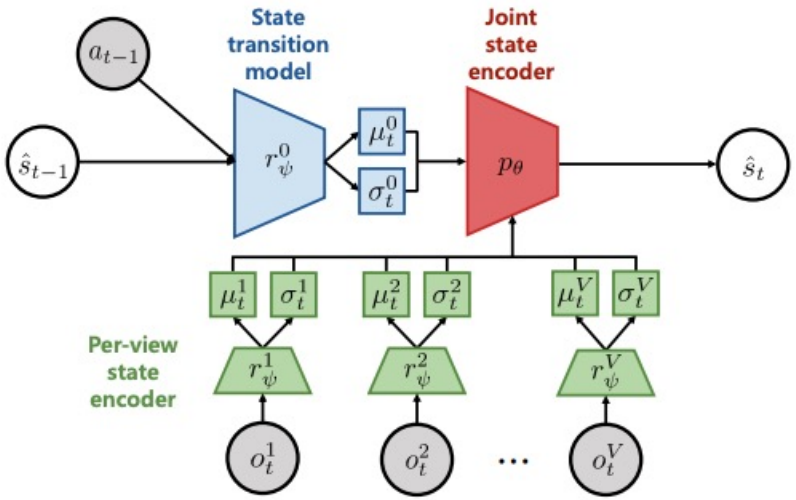


Figure 2. The state encoding process in F2C.

Eq. (10):

1. State transition model: $r^0_\psi(\hat{s}_t \mid \hat{s}_{t-1}, a_{t-1})$
2. Per-view observation model: $q^v_\phi(o^v_t \mid \hat{s}_t)$
3. Joint state encoder: $p_\theta(\hat{s}_t \mid \vec{o}_t, \hat{s}_{t-1}, a_{t-1})$
4. Per-view state encoder: $r^v_\psi(\hat{s}_t \mid o^v_t)$

Fusement

$$p_\theta(\hat{s}_t \mid \vec{o}_t, \hat{s}_{t-1}, a_{t-1}) \triangleq N(\mu_t, \sigma_t^2 \mathbf{I}),$$

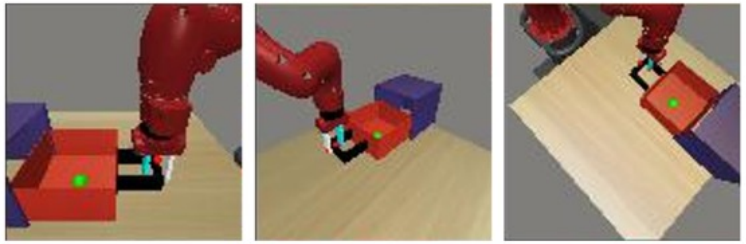
$$\text{where } \mu_t \triangleq \frac{\sum_{v=0}^{V} \mu_t^v/(\sigma_t^v)^2}{\sum_{v=0}^{V} 1/(\sigma_t^v)^2}$$

$$\text{and } \sigma_t^2 \triangleq \frac{1}{\sum_{v=0}^{V} 1/(\sigma_t^v)^2}.$$

Balanced Dependence

$$\sum_{v=1}^{V} \underbrace{\mathbb{E}\left[D_{KL}\left[p_\theta\left(\hat{s}_t|\vec{o}_t, \hat{s}_{t-1}, a_{t-1}\right) \| r^v_\psi\left(\hat{s}_t|o^v_t\right)\right]\right]}$$

- **3** camera views in **Metaworld**



- Result of joint training of representation & policy

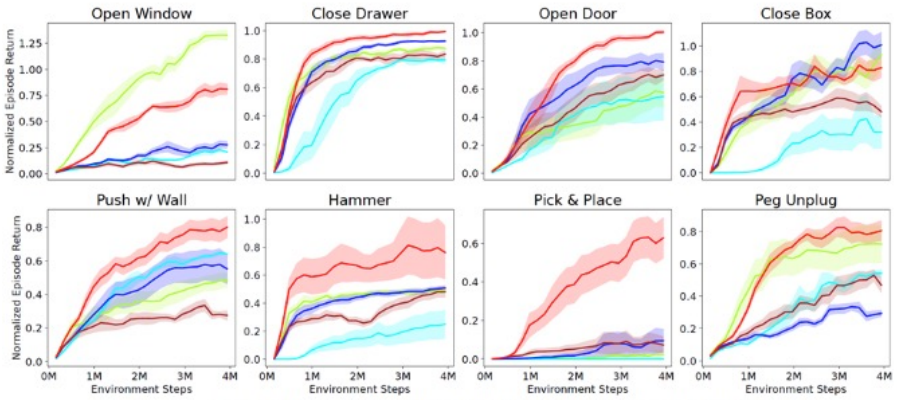


Figure 6. The performance in manipulation tasks given missing views ($\eta = 0.5$).

Learning from Video

Understanding Self-Predictive Learning for Reinforcement Learning

- Self-Predictive Learning is the spectral decomposition on the state transition matrix.
- Representation Collapse is prevented If transition predictor P is optimized faster than the encoder f.

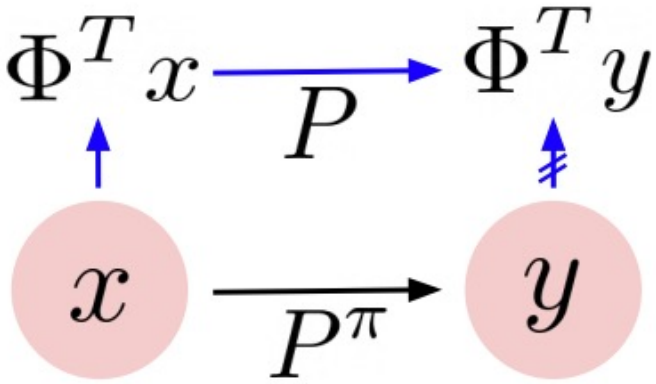


Figure 1. A diagram that outlines the conceptual components of self-predictive learning. The black arrow indicates sampling the transition $y \sim P^\pi(\cdot|x)$. the blue arrow indicates algorithmic components of self-predictive learning: predicting the next state representation $\Phi^T y$ from the first state representation $\Phi^T x$ using prediction matrix P. In practice as shown in Equation (2), self-predictive learning stops the gradient on the prediction target.

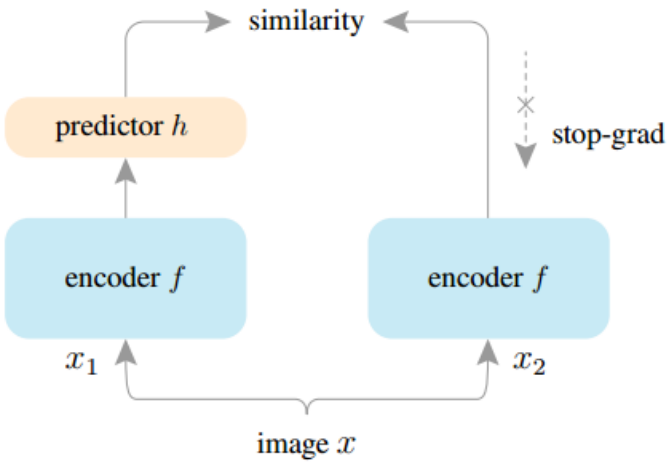


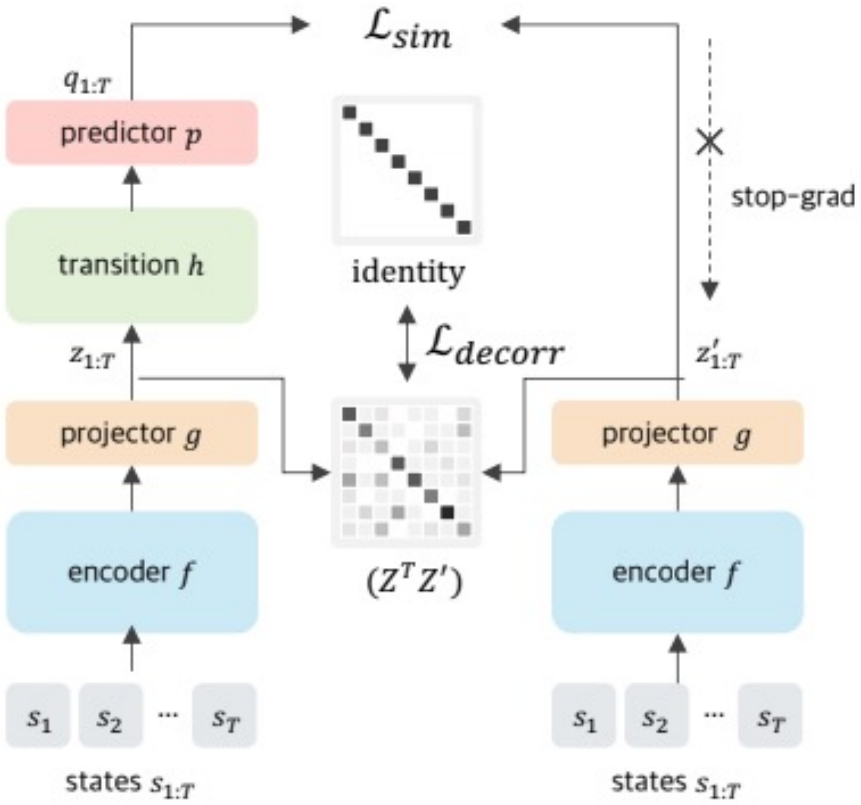
Figure 1. **SimSiam architecture**. Two augmented views of one image are processed by the same encoder network f (a backbone plus a projection MLP). Then a prediction MLP h is applied on one side, and a stop-gradient operation is applied on the other side. The model maximizes the similarity between both sides. It uses neither negative pairs nor a momentum encoder.

[4] Spectral Decomposition Representation for Reinforcement Learning. Ren et al., ICLR 2023.

Learning from Video

SimTPR: On the importance of feature decorrelation in SPR for RL

- GPT-style self-predictive representation learning framework in RL.
- Adopt feature decorrelation objective to prevent collapse to use a powerful transition model.



Causal Similarity Loss

$$\mathcal{L}_{\text{sim}} = -\frac{1}{N(T-k)} \sum_{n=1}^{N} \sum_{t=1}^{T-k} \|\bar{q}_{n,t} - \bar{z}'_{n,t+k}\|_2^2$$

Feature Decorrelation Loss

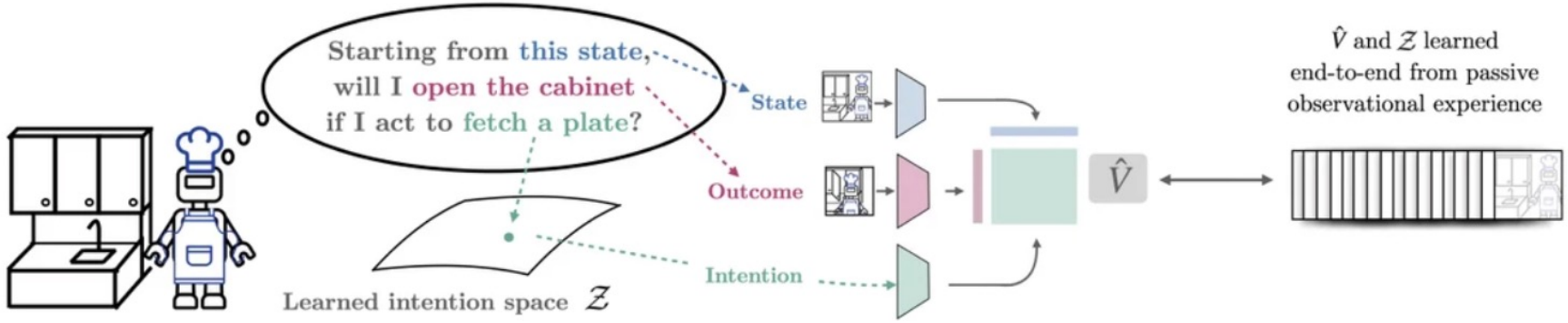
$$\mathcal{L}_{\text{decorr}} = \sum_i (1 - \mathcal{C}_{ii})^2 + \lambda_o \sum_i \sum_{j \neq i} \mathcal{C}_{ij}^2$$

$$\mathcal{C}(\bar{Z}, \bar{Z}')_{i,j} = \frac{\sum_{n=1}^{N} \sum_{t=1}^{T} \bar{z}_{n,t,i} \cdot \bar{z}'_{n,t,j}}{\sqrt{\sum_{n=1}^{N} \sum_{t=1}^{T} (\bar{z}_{n,t,i})^2} \sqrt{\sum_{n=1}^{N} \sum_{t=1}^{T} (\bar{z}'_{n,t,j})^2}}$$

Learning from Video

IVCF: Reinforcement Learning from Passive Data via Latent Intentions

- Learn the expected probability of state visitations given the latent intentions.
- Question: How do we represent the intention? What's the advantage over self-predictive learning?



$$V(s, s_+, z) = \underbrace{\phi(s)^\top}_{\text{State Repr}} \underbrace{T(z)}_{\text{Intent Repr}} \underbrace{\psi(s_+)}_{\text{Outcome Repr}}$$

Learning from Demonstration

ACRO: Principled Offline RL with Rich Exogenous Information

- Propose multi-step inverse model which is invariant to exogenous information.
- Exogenous: control-irrelevant information contained in observations.

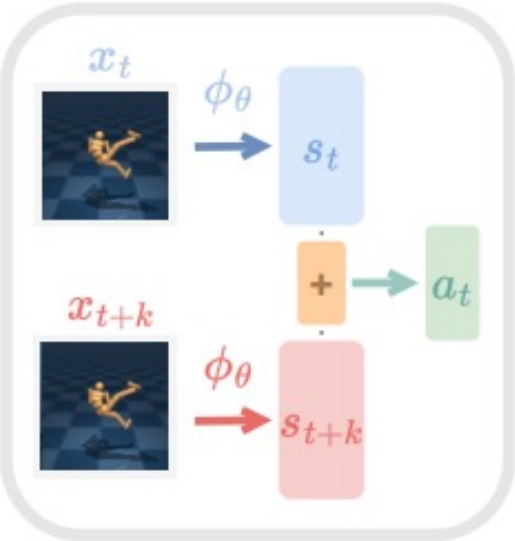


Figure 2: **ACRO** is a multi-step inverse model that predicts the first action conditioned on the current state and the future state. + denotes concatenation.

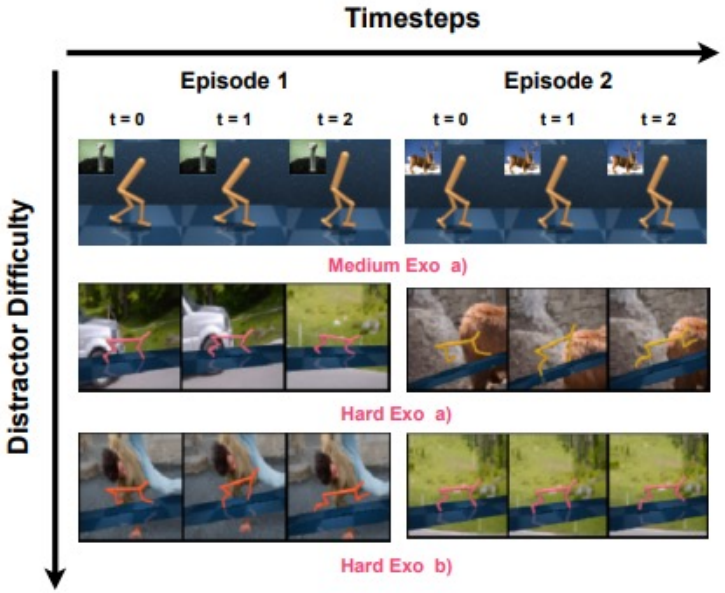


Figure 3: **Examples** of Different Categories of Exogenous Information. Further details of different exogenous information in offline datasets, with visual examples, are provided in Appendix E.

Figure 5: Normalized results across two domains from the v-d4rl distractor suite with varying levels of data shift severity. The easy, medium and hard categories are within the v-d4rl distractor suite of varying shift severity.

Learning from Trajectory

MTM: Masked Trajectory Models for Prediction, Representation, and Control

- Train GPT with a various masking patterns.
- MTM can be used as a forward dynamics, inverse dynamics, and also as an offline RL agent.

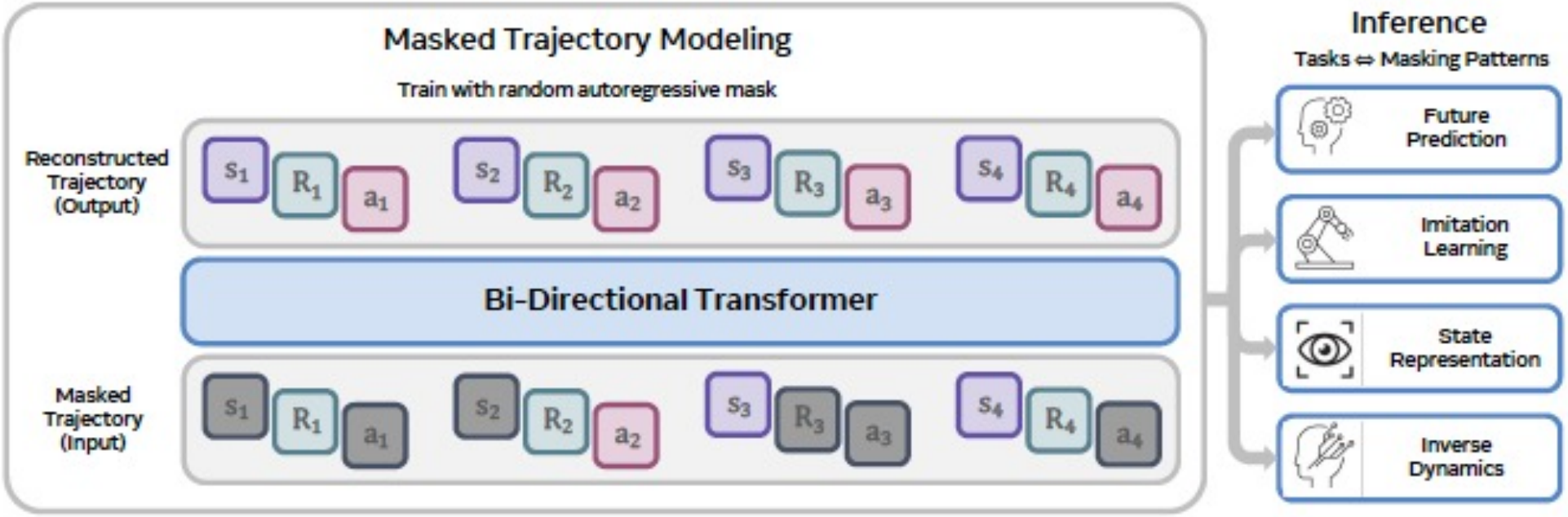


Figure 1. **Masked Trajectory Modeling (MTM) Framework.** (Left) The training process involves reconstructing trajectory segments from a randomly masked view of the same. (Right) After training, MTM can enable several downstream use-cases by simply changing the masking pattern at inference time. See Section 3 for discussion on training and inference masking patterns.

Summary & Future Work

Summary

- Learning Representation from Image: Reconstruction → Object-Centric Reconstruction.
- Learning from Single-View → Multi-View.
- Video: theoretical and empirical success of self-predictive representation learning.
  - However, it's important to note that SPR cannot remove exogenous information.
- Demonstration: multi-step inverse model is proven to be exogenous invariant.
- Trajectory: just GPT…?

Future Work (2023-2024)

- Image: Object Centric objectives. (unsupervised: VICRegL?, supervised: using SAM?).
- Image: 2d (single-view) → 3d (multi-view).
- Video: ViT architecture + SPR objective? Exogenous invariant SPR? Multi-View?
- Trajectory: GPT-style on pixel space (reconstruct latent).

Plasticity

Preliminary: Loss of Plasticity

Definitions of Loss of Plasticity

- When training the model from incremental data, the model exhibits poor generalization performance.

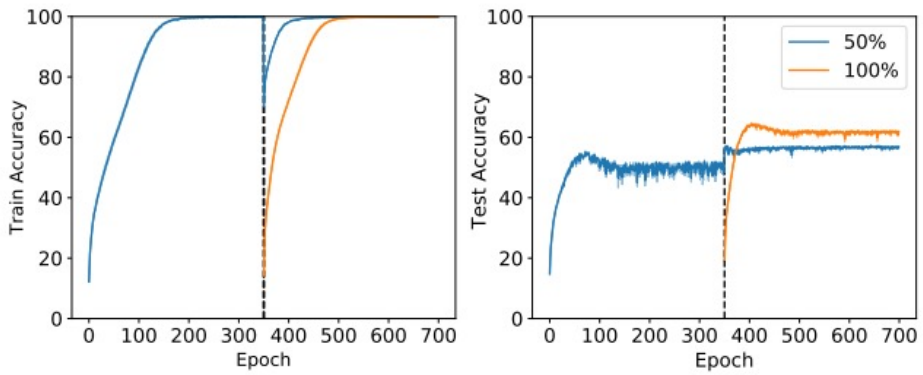


Figure 1: A comparison between ResNets trained using a warm start and a random initialization on CIFAR-10. Blue lines are models trained on 50% of CIFAR-10 for 350 epochs then trained on 100% of the data for a further 350 epochs. Orange lines are models trained on 100% of the data from the start. The two procedures produce similar training performance but differing test performance.

Expression	Definition
Damage from Warm-Starting	[Phenomenon for which] "a warm-started network performs worse on test samples than a network trained on the same data but with a new random initialization"
Damage from Non-Stationarity	"A memory effect where these transient non-stationarities can permanently impact the latent representation and adversely affect generalisation performance"
Capacity Loss	"Reduced ability to fit new targets in deep neural networks"
Loss of Plasticity	"Loss of the ability of the model to keep learning"
Primacy Bias	"A tendency to overfit initial experiences that damages the rest of the learning process"

[5] Shrink & Perturb: On Warm-Starting Neural Network Training. Ash et al., NeurIPS 2020.

[6] Sample-Efficient Reinforcement Learning by Breaking the Replay Ratio Barrier. D'Oro et al., ICLR 2023.

Measuring Plasticity

Understanding Plasticity in Neural Networks

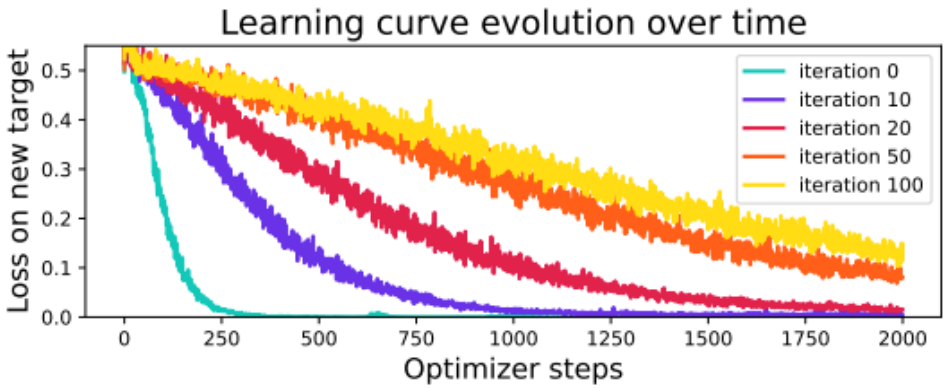
- Measuring Plasticity: Ask the network to learn something new, and see how well it does.
- Lesson: Loss of Plasticity is connected to loss curvature. Let's smooth the curvature via LayerNorm.

3.2. Environments

We construct a simple MDP analogue of image classification, i.e. the underlying transition dynamics are defined over a set of ten states and ten actions, and the reward and transition dynamics depend on whether or not the action taken by the agent is equal to the index of its corresponding state. We

True-label: each state s of the MDP produces an observation from that class in the underlying classification dataset. Given action a, the reward is the indicator function $\delta_{a=s}$. The MDP then randomly transitions to a new state.

Given some offset $a \in \mathbb{R}$, we will apply the transformation $g(x) = a + \sin(10^5 f(\mathbf{x}; \omega_0))$, with ω_0 sampled from the same distribution as θ_0, to construct a challenging prediction objective which measures the ability of the network to perturb its predictions in random directions sampled effectively



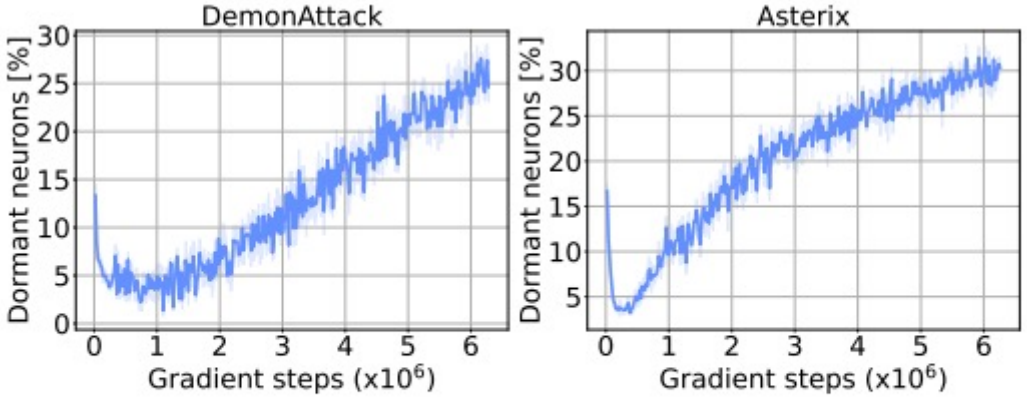
Effect of interventions on plasticity loss

	resnet	transformer	cnn	mlp
two-hot	0.12 ± 0.15	0.05 ± 0.04	0.09 ± 0.17	0.02 ± 0.02
reset last layer	0.00 ± 0.00	0.04 ± 0.07	0.36 ± 0.32	0.10 ± 0.10
weight decay	0.26 ± 0.18	0.17 ± 0.10	0.34 ± 0.08	0.28 ± 0.23
spectral normalization			0.41 ± 0.21	0.43 ± 0.23
layernorm			0.24 ± 0.16	0.25 ± 0.21
shrink and perturb	0.79 ± 0.21	0.23 ± 0.03	0.56 ± 0.20	0.61 ± 0.07
None	0.20 ± 0.21	0.13 ± 0.03	0.31 ± 0.06	0.27 ± 0.21

Measuring Plasticity

The Dormant Neuron Phenomenon in Deep Reinforcement Learning

- Dormant neuron phenomenon: The agent suffers from an increasing number of inactive neurons.
- Propose ReDo which Recycles Dormant neurons throughout training.



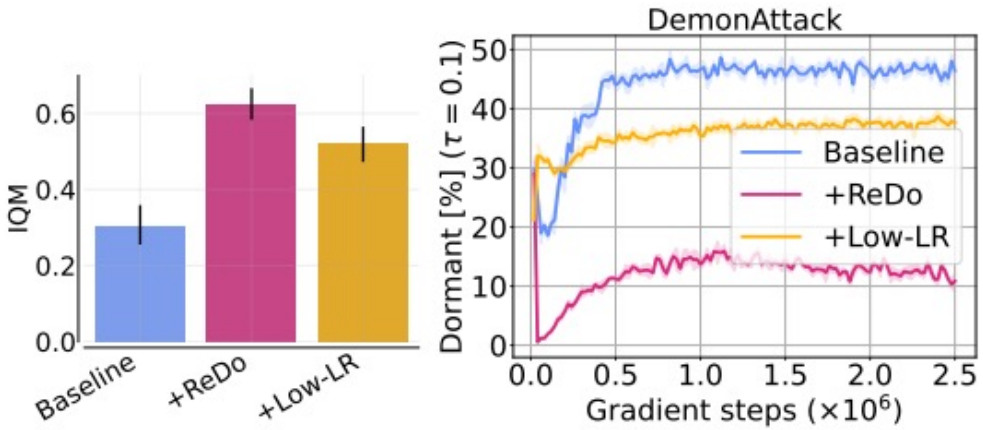
Definition 3.1. Given an input distribution D, let $h_i^\ell(x)$ denote the activation of neuron i in layer ℓ under input $x \in D$ and H^ℓ be the number of neurons in layer ℓ. We define the score of a neuron i (in layer ℓ) via the normalized average of its activation as follows:

$$s_i^\ell = \frac{\mathbb{E}_{x \in D}|h_i^\ell(x)|}{\frac{1}{H^\ell}\sum_{k \in h}\mathbb{E}_{x \in D}|h_k^\ell(x)|} \tag{1}$$

We say a neuron i in layer ℓ is τ-**dormant** if $s_i^\ell \leq \tau$.

Algorithm 1 *ReDo*

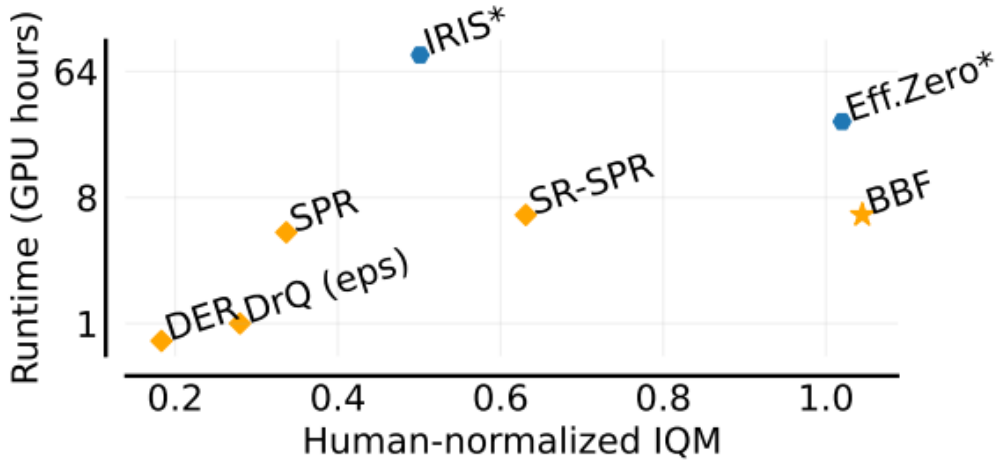
Input: Network parameters θ, threshold τ, training steps T, frequency F
for $t = 1$ to **to** T **do**
  Update θ with regular RL loss
  if $t \mod F == 0$ **then**
    for each neuron i **do**
      if $s_i^\ell \leq \tau$ **then**
        Reinitialize input weights of neuron i

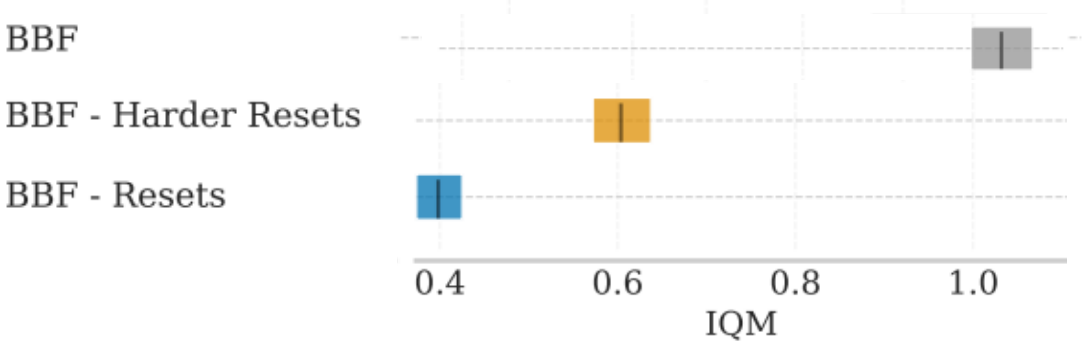


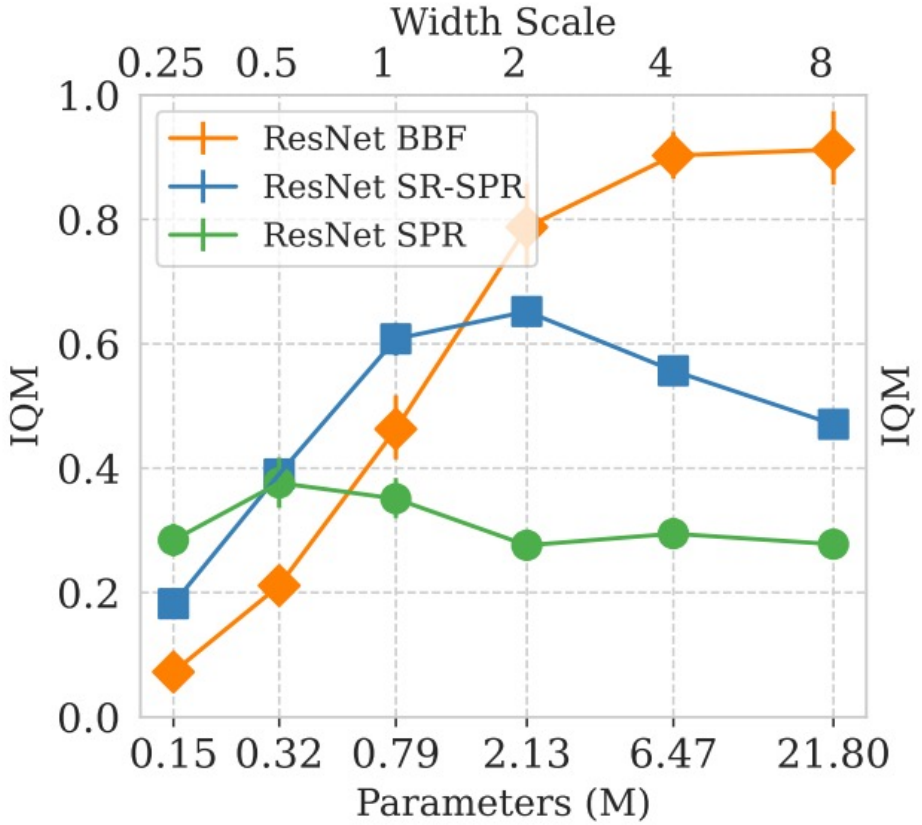
Resolving the Loss of Plasticity

Bigger, Better, Faster: Human-level Atari with human-level efficiency

- Bigger network with reset enable scaling in sample-efficient Atari benchmark.
- Bigger (=Wider), the better performance.







Plasticity ≈ Transferbility?

RankMe: Assessing the Downstream Performance of Pretrained SSL by Their Rank

- Can we assess the performance of pretrained representation without any labels?
- Rank of the pretrained representation is highly correlated to the downstream performance.

Algorithm 1 Hyperparameter selection with RankMe

Require: Models $f_1, \ldots, f_N$ to compare, in increasing value of the hyperparameter

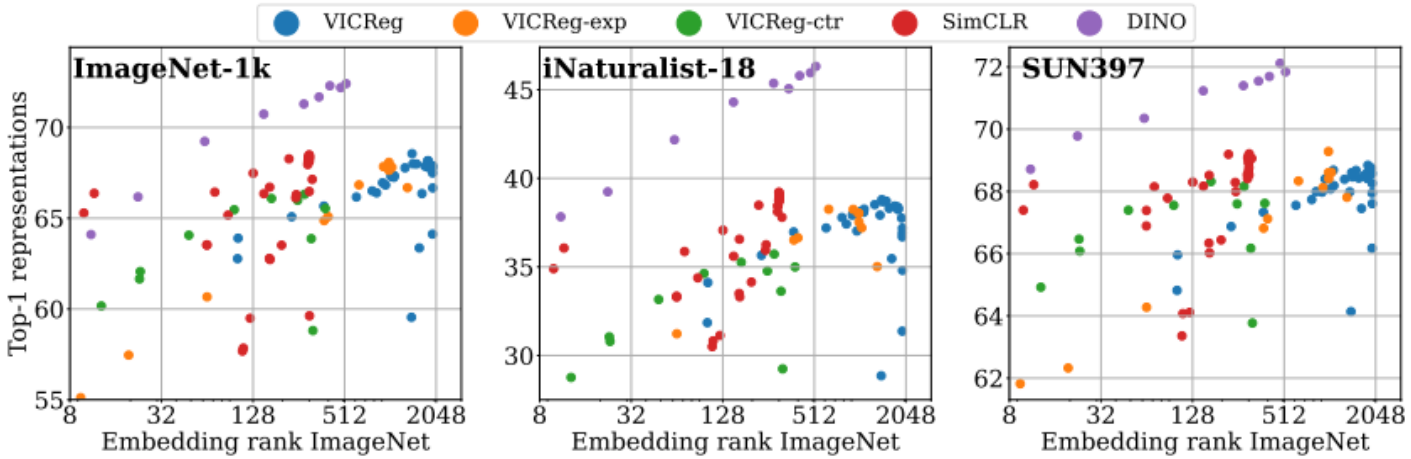
Require: Corresponding ranks $r_1, \ldots, r_N$

$f_{best} \leftarrow f_1,\ r_{best} \leftarrow r_1$
for $i = 2$ to N **do**
  if $r_i > r_{best}$ **then**
    $f_{best} \leftarrow f_i,\ r_{best} \leftarrow r_i$
  else if $r_i = r_{best}$ and $(r_i > r_{i-1}$ or $r_i > r_{i+1})$ **then**
    $f_{best} \leftarrow f_i,\ r_{best} \leftarrow r_i$

σ_k k^{th} singular value of embedding matrix $\boldsymbol{Z}$

$$\text{RankMe}(\boldsymbol{Z}) = \exp\left(-\sum_{k=1}^{\min(N,K)} p_k \log p_k\right)$$

$$\text{with} \quad p_k = \frac{\sigma_k(\boldsymbol{Z})}{\|\sigma(\boldsymbol{Z})\|_1} + \epsilon,$$



Dataset	Method	Labels	VICReg				SimCLR			DINO	
			cov.	inv.	LR	WD	temp.	LR.	WD.	t-temp.	s-temp.
ImageNet	ImageNet Oracle	✓	68.2	68.2	68.6	68.0	68.5	68.5	68.3	72.3	72.4
	α-ReQ	X	**67.9**	67.5	59.5	**67.8**	63.5	**68.1**	32.3	71.7	66.2
	RankMe	X	67.8	**67.9**	**68.2**	**67.8**	**67.1**	68.0	**68.3**	**72.2**	**72.4**
OOD	ImageNet Oracle	✓	68.7	68.7	68.9	68.8	68.7	68.7	68.8	71.9	72.5
	α-ReQ	X	**68.1**	67.8	63.8	**68.4**	65.1	68.2	68.6	**71.8**	68.5
	RankMe	X	67.7	**68.3**	**68.7**	**68.4**	**67.6**	**68.4**	**68.8**	**71.8**	**72.5**

Summary & Future Work

Summary

- Plasticity: Network's ability to adapt.
- Plasticity can be important in transfer / continual / reinforcement learning.
- Two types of non-stationarity to handle
  - Input non-stationarity: marginal distributional shift / out-of-distributional shift.
  - Target non-stationarity: change of label / TD-learning.

Future Work (2023-2024)

- Thorough investigation of the loss of plasticity based on different types of non-stationarity.
- Deep neural network loves lower-rank [7] ≈ increase in dormant neurons?
- Recycle may not work if feature embeddings' eigen spectrum is heavily skewed.
- Feature-wise rank maximization can be an answer…? (smoother curvature).

[7] The Low-Rank Simplicity Bias in Deep Networks. Huh et al (w/ Richard Zhang, Phillip Isola)., TMLR 2023.

Others

General Computer Vision

Scaling Vision Transformers to 22 Billion Parameters

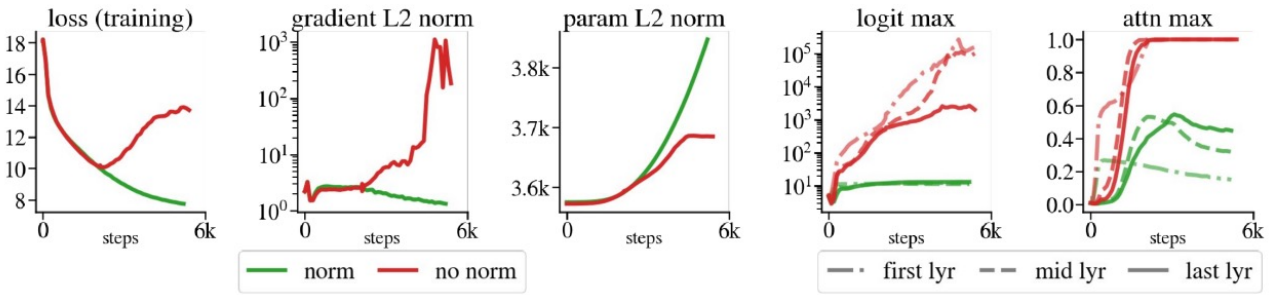
- Let's scale VIT on JFT (4B images).
- Bigger the Better!

Instabilities past ~8B params

- Uncontrolled attention logit growth
- Fix: *QK Normalization*

Query LayerNorm Key LayerNorm

$$\text{softmax}\left[\frac{1}{\sqrt{d}}\text{LN}(XW^Q)(\text{LN}(XW^K))^T\right]$$



Linear eval

Excellent, especially on OOD, almost matches fine-tuning (89.5% ImageNet @224x224)

Classification / OOD

ImageNet saturating, significant improvements on OOD evaluations

Video

Competitive results using per-frame representations (88% Kinetics 400)

Contrastive zero-shot

Outperforms ViT-e/g, a bit worse than SOTA on ImageNet, but better OOD (87.6% ObjectNet)

Dense prediction

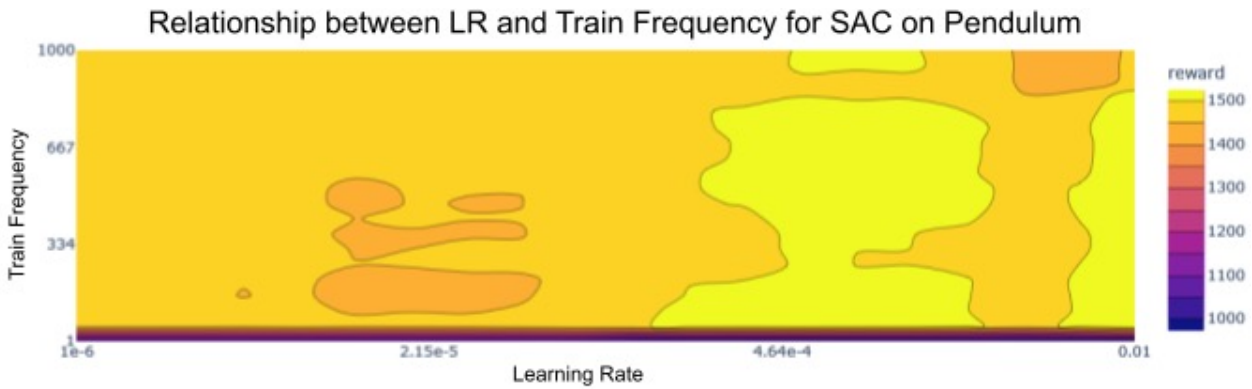
Competitive (not SOTA) with frozen tower on semantic segmentation, monocular depth estimation

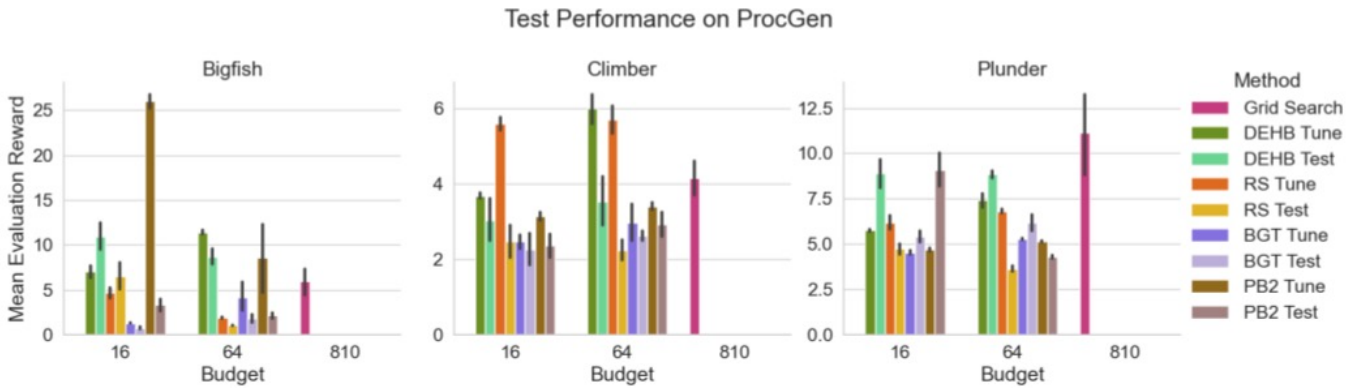
Hyperparameter Optimization

Hyperparameters in Reinforcement Learning and How to Tune Them

- Hyperparameter choices in RL can significantly affect the agent's final performance.
- Let's use SOTA hyperparameter optimization (HPO) algorithms to tune RL.

1. Hyperparameter sweeps across all exposed hyperparameters of 3 different RL algorithms classes (PPO, SAC, DQN)
2. Tuning the algorithms across different search space sizes and training seeds:
   - Random Search (RS) [Bergstra & Bengio, 2012]
   - Population Based Bandits (PB2) [Parker-Holder et al., 2021]
   - DE Hyperband (DEHB) [Awad et al. 2021]
3. Tuning the state-of-the-art agents of two popular benchmarks, ProcGen and Brax, using **small budgets of 16 and 64 total training runs**





Q & A